

Deep Dive into the C Compilation Process: From Source Code to Machine Execution

Project: ccomp

GitHub: <https://github.com/lrisguan/ccomp>

Author: ccomp contributors

2026-02-23

Contents

1	How Computers Execute Programs	5
1.1	Purpose	5
1.2	Outline	5
1.3	Content Plan	5
1.4	Key Terms	6
1.5	Diagrams Planned	6
1.6	Experiment	6
2	C Compilation Overview	7
2.1	Purpose	7
2.2	Outline	7
2.3	Content Plan	7
2.4	Key Terms	7
2.5	Diagrams Planned	8
2.6	Experiment	8
3	Preprocessing	9
3.1	Purpose	9
3.2	Outline	9
3.3	Content Plan	9
3.4	Key Terms	9
3.5	Experiment	10
4	Compilation Internals	11
4.1	Purpose	11
4.2	Outline	11
4.3	Content Plan	11
4.4	Key Terms	12
4.5	Experiment	12
5	Assembly and Object Files	13
5.1	Purpose	13
5.2	Outline	13
5.3	Content Plan	13
5.4	Key Terms	13
5.5	Experiment	14

6	ELF Format Deep Dive	15
6.1	Purpose	15
6.2	Outline	15
6.3	Content Plan	15
6.4	Key Terms	15
6.5	Diagrams Planned	16
6.6	Experiment	16
7	Linking and Relocation	17
7.1	Purpose	17
7.2	Outline	17
7.3	Content Plan	17
7.4	Key Terms	17
7.5	Experiment	18
8	Static and Dynamic Libraries	19
8.1	Purpose	19
8.2	Outline	19
8.3	Content Plan	19
8.4	Key Terms	19
8.5	Experiment	20
9	Standard Library and Syscalls	21
9.1	Purpose	21
9.2	Outline	21
9.3	Content Plan	21
9.4	Key Terms	21
9.5	Experiment	22
10	ABI and Cross-Platform	23
10.1	Purpose	23
10.2	Outline	23
10.3	Content Plan	23
10.4	Key Terms	23
10.5	Experiment	24
11	Process Loading and Memory Layout	25
11.1	Purpose	25
11.2	Outline	25
11.3	Content Plan	25
11.4	Key Terms	25
11.5	Experiment	26

Chapter 1

How Computers Execute Programs

1.1 Purpose

Establish the CPU and memory execution model that motivates later compilation details.

1.2 Outline

1. CPU execution model and architectural state
2. Instruction cycle: fetch, decode, execute
3. Registers and register file roles
4. Memory model and addressing
5. Stack and function call mechanics
6. Calling conventions overview
7. Transition to ABI discussion

1.3 Content Plan

- CPU execution model: instructions update architectural state and memory.
- Instruction cycle: control flow, sequencing, and side effects.
- Registers: general purpose, PC, SP, flags, callee vs caller saved.
- Memory model: byte addressing, alignment, endianness.
- Stack mechanics: frame layout, prologue and epilogue, return address.
- Calling conventions: argument passing, return values, stack cleanup.
- ABI bridge: why a stable contract is required.

1.4 Key Terms

- instruction pointer
- stack frame
- calling convention
- ABI

1.5 Diagrams Planned

CPU state and instruction cycle diagram placeholder.

Figure 1.1: Planned CPU state and instruction cycle diagram.

Stack frame layout diagram placeholder.

Figure 1.2: Planned stack frame layout diagram.

1.6 Experiment

See `how-computers-execute-programs-exp/how-computers-execute-programs-exp.tex` in this chapter.

Chapter 2

C Compilation Overview

2.1 Purpose

Provide a complete map of the compilation pipeline from source to executable.

2.2 Outline

1. Pipeline stages and artifacts
2. .c to .i preprocessing
3. .i to .s compilation
4. .s to .o assembly
5. .o to executable linking
6. Toolchain components and roles

2.3 Content Plan

- Pipeline overview with artifact boundaries.
- Stage responsibilities: compiler, assembler, linker.
- Common drivers: gcc, clang, as, ld.
- Why separation matters for debugging and optimization.

2.4 Key Terms

- compilation pipeline
- translation unit
- assembler
- linker

End-to-end pipeline diagram placeholder.

Figure 2.1: Planned compilation pipeline diagram.

2.5 Diagrams Planned

2.6 Experiment

See `c-compilation-overview-exp/c-compilation-overview-exp.tex` in this chapter.

Chapter 3

Preprocessing

3.1 Purpose

Explain preprocessing as a textual substitution stage that builds compilation units.

3.2 Outline

1. Macro expansion model
2. Include mechanism
3. Conditional compilation
4. Header guards
5. Compilation units and translation units

3.3 Content Plan

- Treat preprocessing as a deterministic text transformation.
- Macro expansion order, function-like macros, and pitfalls.
- Include search paths and include graphs.
- Conditional blocks and feature toggles.
- Header guards and include once behavior.
- How preprocessing defines a compilation unit boundary.

3.4 Key Terms

- macro expansion
- include path
- header guard
- translation unit

3.5 Experiment

See preprocessing-exp/preprocessing-exp.tex in this chapter.

Chapter 4

Compilation Internals

4.1 Purpose

Detail how source code becomes optimized machine-oriented assembly.

4.2 Outline

1. Lexical analysis
2. Parsing and grammar
3. AST construction
4. Intermediate representation (IR)
5. Optimization passes
6. Code generation
7. Optimization flags (-O0, -O1, -O2, -O3)

4.3 Content Plan

- Tokenization and error reporting.
- Parser output and syntax trees.
- AST vs IR roles and transformations.
- Optimization pipeline: local, global, SSA-based.
- Code generation: instruction selection, register allocation.
- Mapping high-level constructs to assembly idioms.
- Flag effects on debugability and performance.

4.4 Key Terms

- AST
- IR
- SSA
- register allocation

4.5 Experiment

See `compilation-internals-exp/compilation-internals-exp.tex` in this chapter.

Chapter 5

Assembly and Object Files

5.1 Purpose

Explain how assembly is turned into relocatable object files and what those objects contain.

5.2 Outline

1. Role of the assembler
2. Object file structure overview
3. Symbol table basics
4. Section layout (.text, .data, .bss, .rodata)
5. Relocatable object semantics

5.3 Content Plan

- Assembler responsibilities and encoding rules.
- Object file as a container for code, data, symbols, and relocations.
- Symbol table entries and binding.
- Section semantics and how the linker interprets them.
- Relocatable vs executable differences at this stage.

5.4 Key Terms

- object file
- section
- symbol table
- relocation

5.5 Experiment

See `assembly-and-object-files-exp/assembly-and-object-files-exp.tex` in this chapter.

Chapter 6

ELF Format Deep Dive

6.1 Purpose

Provide a detailed, field-by-field understanding of the ELF format.

6.2 Outline

1. ELF header
2. Section header table
3. Program header table
4. Symbol table and string tables
5. Relocation entries
6. Static vs dynamic ELF

6.3 Content Plan

- Identify magic bytes, class, endianness, and ABI fields.
- Section headers: names, offsets, sizes, and flags.
- Program headers: loadable segments and memory mapping.
- Symbol tables: bindings, visibility, and section index.
- Relocations: addends, types, and targets.
- Differences between static and dynamic binaries.

6.4 Key Terms

- ELF header
- section header table
- program header table
- relocation

6.5 Diagrams Planned

ELF file layout diagram placeholder.

Figure 6.1: Planned ELF file layout diagram.

Section vs program header mapping diagram placeholder.

Figure 6.2: Planned section vs program header mapping diagram.

6.6 Experiment

See `elf-format-deep-dive-exp/elf-format-deep-dive-exp.tex` in this chapter.

Chapter 7

Linking and Relocation

7.1 Purpose

Explain how the linker resolves symbols and applies relocations.

7.2 Outline

1. Symbol resolution rules
2. Strong vs weak symbols
3. Relocation process
4. Static linking flow
5. Library search order
6. Common linking errors

7.3 Content Plan

- Resolution across multiple object files.
- Weak symbol precedence and overrides.
- Relocation entries and address fixing.
- Static linking of archives and object extraction.
- Search order for -L and -l.
- Typical errors: undefined, multiple definition, missing libs.

7.4 Key Terms

- symbol resolution
- relocation
- weak symbol
- archive

7.5 Experiment

See `linking-and-relocation-exp/linking-and-relocation-exp.tex` in this chapter.

Chapter 8

Static and Dynamic Libraries

8.1 Purpose

Compare static and dynamic libraries and explain runtime resolution mechanisms.

8.2 Outline

1. Structure of .a archives
2. Structure of .so shared objects
3. Dynamic loader role
4. PLT and GOT
5. Lazy binding
6. Runtime symbol resolution

8.3 Content Plan

- Archive indexing and object extraction.
- PIC and shared object metadata.
- Loader responsibilities and search paths.
- PLT/GOT indirection and lazy binding.
- Symbol interposition and versioning considerations.

8.4 Key Terms

- archive
- shared object
- PLT
- GOT

8.5 Experiment

See `static-and-dynamic-libraries-exp/static-and-dynamic-libraries-exp.tex` in this chapter.

Chapter 9

Standard Library and Syscalls

9.1 Purpose

Distinguish C standard library responsibilities from kernel system calls.

9.2 Outline

1. C standard library vs system calls
2. glibc role and layering
3. User mode vs kernel mode
4. syscall instruction path
5. strace at a conceptual level

9.3 Content Plan

- Library wrappers and ABI compliance.
- syscall numbers, calling conventions, and return values.
- Error reporting via errno.
- Why libc adds buffering and higher-level APIs.
- Observing syscalls without oversimplifying the boundary.

9.4 Key Terms

- libc
- syscall
- user mode
- kernel mode

9.5 Experiment

See `standard-library-and-syscalls-exp/standard-library-and-syscalls-exp.tex` in this chapter.

Chapter 10

ABI and Cross-Platform

10.1 Purpose

Explain ABI contracts and cross-platform compilation differences.

10.2 Outline

1. What is an ABI
2. Calling conventions in x86_64 vs AArch64
3. Binary compatibility constraints
4. ELF vs PE
5. Linux vs Windows toolchains
6. Cross-compilation toolchains

10.3 Content Plan

- ABI as a stable binary interface.
- Register and stack argument passing differences.
- Structure layout, alignment, and endianness effects.
- ELF and PE loader expectations.
- Cross toolchain selection and sysroot use.

10.4 Key Terms

- ABI
- calling convention
- sysroot
- cross-compilation

10.5 Experiment

See `abi-and-cross-platform-exp/abi-and-cross-platform-exp.tex` in this chapter.

Chapter 11

Process Loading and Memory Layout

11.1 Purpose

Describe how the OS loads an executable and maps it into virtual memory.

11.2 Outline

1. Loader responsibilities
2. Virtual memory fundamentals
3. Text, data, heap, stack layout
4. Dynamic linker loading
5. mmap and shared libraries

11.3 Content Plan

- Loader reads program headers and maps segments.
- Virtual memory pages, permissions, and ASLR.
- Typical process layout and growth directions.
- Dynamic linker responsibilities during startup.
- mmap usage for shared objects and anonymous memory.

11.4 Key Terms

- virtual memory
- program header
- dynamic linker
- mmap

11.5 Experiment

See process-loading-and-memory-layout-exp/process-loading-and-memory-layout-exp.tex in this chapter.